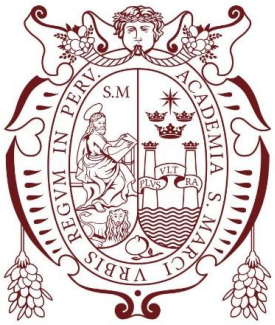
NATIONAL UNIVERSITY OF SAN MARCOS



UNMSM

Doctoral Program in Deep Tech, Artificial Intelligence & Emerging Technologies

Faculty of Systems Engineering and Informatics

ASSIGNMENT N° 6

Reproducibility Audit Report

Professor	Dr. Loveleen Gaur	
Student	Acosta Ticona, Angela Milén	COD. 26207270

Lima 2026

Method. The audit applies the Session 6 reproducibility scorecard (9 items), framed by the REFORMS recommendations for machine-learning-based science (Kapoor et al., 2024). Every claim below cites a page of the paper, a file of the authors' repository, or a log from my own re-execution, which anyone can repeat with `run_audit.sh`.

Verdict

Overall score: 7 / 18 (39%) — partially reproducible. The design is sound (temporal split, model selection on a validation set, risk features learned from training data only), but the results cannot be verified as published and the code does not reproduce itself.

Three findings drive the score:

1. **No seeds anywhere in the main pipeline.** Re-running the authors' unchanged training code five times on the released synthetic data gave Precision@1% from **15.8% to 34.7%** (mean 26.1% ± 8.1 SD). Adding a seed made a run exactly repeatable (verified: seed 0 run twice gave identical metrics). The paper reports averages over 20 runs but never their spread.
2. **The environment is not fixed.** `requirements.txt` lists package names with no versions. The code runs unchanged in a 2020-era environment, but fails twice with current libraries (pandas ≥ 2.0, PyTorch ≥ 2.6), and even the deterministic XGBoost baseline changes with the library version (AUC 0.730 vs 0.723 on identical data).
3. **No statistical evidence for the headline comparison.** DATE is compared with six baselines by point estimates only: no standard deviations, confidence intervals or significance tests. The real Nigerian data are under a non-disclosure agreement, so the published numbers cannot be checked.

1. Paper metadata

Field	Detail
Citation (APA 7)	Kim, S., Tsai, Y.-C., Singh, K., Choi, Y., Ibok, E., Li, C.-T., & Cha, M. (2020). DATE: Dual attentive tree-aware embedding for customs fraud detection. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining* (pp. 2880–2890). ACM. https://doi.org/10.1145/3394486.3403339
Venue type	Peer-reviewed conference (KDD, Applied Data Science track)
Domain	Customs fraud detection on transaction-level import declarations
Code	github.com/Roytsai27/Dual-Attentive-Tree-aware-Embedding, linked from the paper (p. 10)
Version audited	Commit `5c540d6` (18 July 2021), the latest on the default branch at the audit date; 106 commits since February 2020
Licence	CC BY-NC-SA 4.0

Field	Detail
Data in the paper	1,932,151 Nigerian import declarations, 2013–2017 (p. 3); not public, under a non-disclosure agreement (p. 10)
Data in the repository	Synthetic file of 100,000 declarations, January–December 2013, generated with CTGAN (p. 10)
Audit date	16 October 2026

2. Experimental setup

Element	What the paper reports	Page
Task	Rank import declarations by fraud risk: binary illicitness plus predicted extra tax revenue	4
Model	Three stages: XGBoost (100 trees, depth 4) produces leaf "cross features"; dual attention over leaves, importer ID and HS code; joint loss for classification and revenue ($\alpha = 10$, $\lambda = 0.01$)	4–6
Baselines	Price, Importer, Isolation Forest, GBDT (XGBoost), GBDT + LR, TEM	6
Split	Temporal. Train 2013–2016: 1,635,157 rows (84.4%); validation: last month before the test year, 25,948 rows (1.3%); test 2017: 276,440 rows (14.3%). Illicit rates 3.94%, 2.51%, 2.24%	6
Model selection	Best epoch saved on the validation set; prediction after epoch 5. Classification threshold chosen on validation	6; code `train.py`, `generate_loader.py`
Metrics	Precision@n%, Recall@n%, Revenue@n% at n = 1, 2, 5, 10; AUC; best F1 over thresholds 0.1–0.6	6
Runs	Main results "averaged over 20 runs"; hyperparameter analyses "averaged over ten repeated experiments"	6, 10
Extra analyses	Ablation (Table 4), training length (Fig. 3), subgroups by importer and HS frequency (Table 5), case studies (Table 6), robustness to value roll-back (Table 7)	7–9

The headline claim is **92.7% precision and 49.3% revenue recall when inspecting the top 1%** of declarations (abstract; Table 3, p. 6). Note that the two figures come from two different outputs of the model: the 92.7% precision from DATE-CLS and the 49.3% revenue from DATE-REV. No single configuration achieves both at once.

3. Leakage check

No test-set leakage was found in the code; the one material risk is post-inspection values in the original features, which the authors disclose.

Check	Finding	Evidence	Status
Split order and time	Split is temporal (test year after training years), so no future rows enter training	p. 6; `preprocess_data.py` splits by `sgd.date`	OK
Risk indicators (target-based features)	Importer, declarant, HS and office risk flags are computed from **training rows only** and then applied to validation and test	p. 3; `find_risk_profile(train, ...)` in `preprocess_data.py`	OK
Unseen importers and HS codes	Mapped to a zero embedding, never fitted on test labels	p. 5; `importer_mapping.get(x, 0)` in `generate_loader.py`	OK
Thresholds and model selection	Chosen on the validation set, then applied to test	`train.py`, `generate_loader.py`	OK
Post-inspection values	For illicit cases, the authors used the values **adjusted by customs after inspection** (fob, cif, total taxes), because the initial declared values could not be extracted. Illicit rows therefore carry information created by the outcome. Illicit transactions show values 66% higher on average	p. 8 ("Challenges"); robustness test in Table 7, p. 9	**Risk**
In-sample target encoding	Risk flags for **training** rows use those same rows' labels, which can make XGBoost over-trust these features. This does not touch the test set	`preprocess_data.py`	Minor
Same importer in train and test	96.2% of test declarations in the synthetic data come from importers already seen in training. With a temporal split this mirrors deployment, so it is realistic rather than leakage; Table 5 usefully reports results for new importers separately	p. 7–8; own computation	OK, documented

Assessment. The pipeline itself is clean. The authors' roll-back experiment (multiplying values by 0.6) is a partial mitigation only: it shows the model still works when the gap is removed, but the headline Table 3 remains computed on the adjusted, outcome-informed values.

4. Statistical rigor

The paper averages many runs but reports no spread, interval or test, so none of its model comparisons can be judged against noise.

Question	Finding	Page
Many runs?	Yes: main results averaged over 20 runs; hyperparameter curves over 10	6, 10
Spread reported?	**No.** No SD, min–max or error bars in Tables 3–7 or Figures 3–8	6–10
Seeds reported?	**No.** Neither the paper nor the main code sets or names a seed	—
Statistical test for model comparisons?	**No** test is named	6–7
Effect sizes and confidence intervals?	Differences are given as point estimates only; no intervals	6–7
Single held-out test set?	Yes, one fixed test year, so a bootstrap interval or McNemar's test on paired predictions would have been feasible	6

Why this matters. Several claimed gains are small compared with plausible run-to-run variation:

Comparison (Table 3, p. 6)	DATE-CLS	Best baseline	Gap
Precision@1% vs GBDT + LR	92.66%	90.95%	+1.71 points
Precision@1% vs TEM	92.66%	88.72%	+3.94 points
AUC vs TEM	96.79%	96.52%	+0.27 points

In my re-execution (Section 5), the SD of Precision@1% across five unseeded runs was **8.1 points** on the synthetic data. That data set is about 19 times smaller than the original, so the real-data variance is probably smaller, but without a reported SD a reader cannot tell whether gaps of 0.3 to 4 points exceed the noise. A paired comparison over the same runs, with a corrected test (Nadeau & Bengio, 2003) or bootstrap intervals, would settle it.

5. Re-execution

The repository ran end to end only after two one-line fixes, and the same code then gave a different result on every run.

Environment. Linux, 2 CPU cores, no GPU; Python 3.12.3; PyTorch 2.14.1 (CPU); pandas 3.0.6; NumPy 2.5.4; scikit-learn 1.9.1; XGBoost 3.4.1; Ranger optimizer installed from its GitHub repository (commit `c82a304`). The authors used Python 3.7 (README); no exact versions are given, so current releases were used, as a stranger would.

Steps, as in the README: `python preprocess_data.py; python generate_loader.py; python train.py` on the bundled synthetic data.

Step	Result	Cause	Fix applied
1. `preprocess_data.py`	**Failed:** `AttributeError: 'DatetimeProperties' object has no attribute 'weekofyear'`	`Series.dt.weekofyear` was removed in pandas 2.0	`dt.isocalendar().week.astype(int)`
2. `generate_loader.py`	Ran (XGBoost and GBDT + LR baselines)	—	—
3. `train.py --device cpu`	Trained 5 epochs, then **failed** at evaluation: `UnpicklingError: Weights only load failed`	PyTorch 2.6 changed the default of `torch.load` to `weights_only=True`	`torch.load(model_path, weights_only=False)`, safe here because the file was created locally

Run time: about 170 seconds per DATE training run on 2 CPU cores.

Five runs of the unchanged DATE training code (synthetic test set: 19,516 declarations, November–December 2013):

Run	Precision@1%	Precision@10%	Revenue@10%	AUC	F1
1	30.10%	25.46%	35.35%	0.785	0.324
2	34.69%	28.74%	39.92%	0.809	0.333
3	30.61%	28.33%	40.86%	0.812	0.366
4	19.39%	26.54%	37.45%	0.804	0.159
5	15.82%	24.08%	33.93%	0.800	0.141
Mean ± SD	**26.1% ± 8.1**	**26.6% ± 2.0**	**37.5% ± 2.9**	**0.802 ± 0.011**	**0.265 ± 0.106**

For reference, the XGBoost baseline from step 2 (deterministic with its default seed) reached Precision@1% 15.3%, Precision@10% 20.2% and AUC 0.723.

Interpretation.

- On the synthetic data, DATE beats XGBoost on average, which is consistent with the paper's direction of effect.
- The spread is large: Precision@1% more than doubles between the worst and best run, and the worst DATE run is barely above the XGBoost baseline. F1 varies by a factor of 2.6 because the threshold is re-selected on validation in each run.
- Absolute values are far below the paper's (for example 26% vs 92.7% Precision@1%). This is expected, because the synthetic file is a small, single-year CTGAN sample, so it **cannot** be used to verify the published numbers. It only verifies that the code runs and how unstable it is.

5.1 Broken code or changed environment?

To separate the two, the **unmodified** code was also run in a 2020-era environment: Python 3.8.20 (Python 3.7, which the authors used, was not available), PyTorch 1.4.0, pandas 1.0.5, NumPy 1.18.5, scikit-learn 0.22.2, XGBoost 1.0.2.

Check	2020-era libraries	Current libraries
Runs without changes?	**Yes**, all three steps (only a headless plotting backend, `MPLBACKEND=Agg`, was needed)	**No**, two failures (Section 5)
XGBoost baseline AUC (deterministic, same data)	0.730	0.723
XGBoost baseline Precision@1%	19.9%	15.3%
One DATE run, Precision@1% / AUC	32.1% / 0.8125	15.8–34.7% / 0.785–0.812 (5 runs)
Run time per DATE run	about 350 s (shared CPU)	about 170 s

Conclusion. The code was correct for its time; it broke because the authors did not pin versions. Version drift also changes results silently: the same deterministic baseline on the same data moves by 0.8 AUC points (0.7302 vs 0.7225) and 4.6 precision points.

5.2 Does fixing the seed solve the repeatability problem?

A small optional argument (`--seed`, patch `03_add_seed_option.patch`) seeds Python, NumPy and PyTorch. With it, **seed 0 run twice gave identical metrics**, so each run becomes repeatable. Across seeds 0–4, Precision@1% ranged from 28.1% to 32.6% (29.9% ± 1.7 SD); this spread is the run-to-run variation a paper should report.

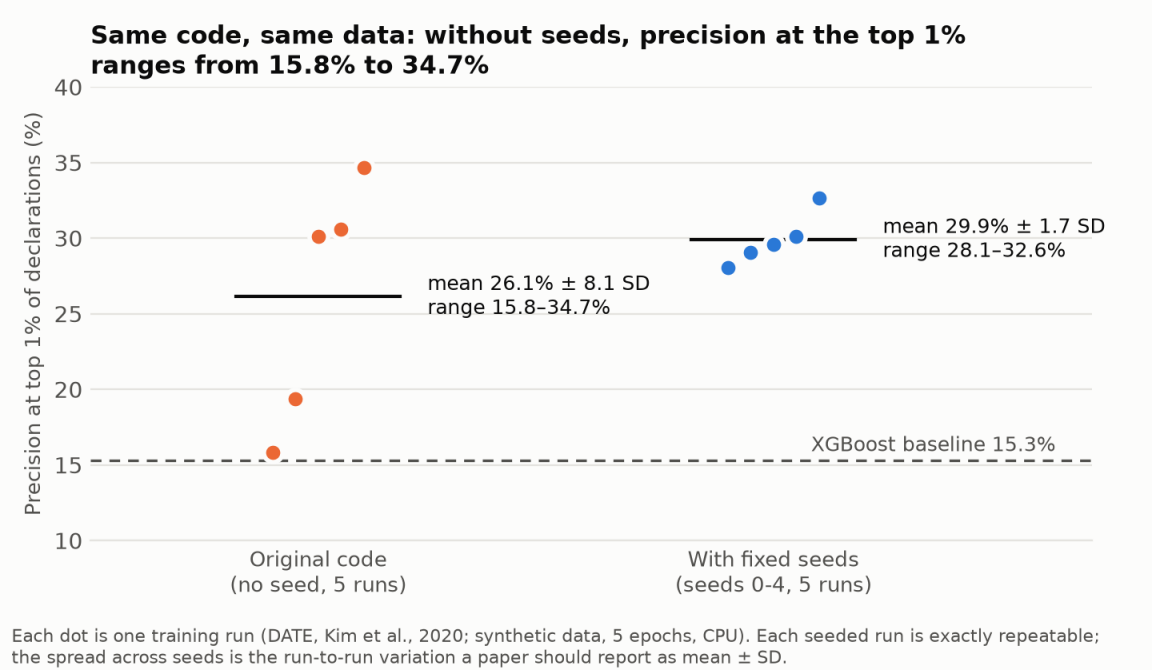


Figure 1. Each dot is one training run of the unchanged DATE model on the synthetic data (5 epochs, CPU). The dashed line is the XGBoost baseline in the same environment. Five runs per group is a small sample, so the narrower spread of the seeded group is not evidence that seeding reduces variance; it only shows the variance is real and must be reported.

6. Missing information

A stranger would need the following to reproduce the published results, and none of it is provided:

- **Exact environment:** package versions (the `requirements.txt` lists names only), CUDA version and operating system. Only "Python 3.7" is stated.
- **Seeds:** for XGBoost, PyTorch, NumPy and the shuffled data loader.
- **Variability:** SD or range for the 20-run averages and the 10-run hyperparameter curves.
- **Compute:** hardware, GPU model, training time per run, and total compute.
- **Data:** the real data cannot be shared (NDA), and there is no access route for qualified researchers. The synthetic file covers one year and 100,000 rows, versus five years and 1.9 million rows in the paper, so it cannot reproduce Table 3.
- **Validation set:** the text says "the last month worth of data from the previous four years", but its size (25,948 rows, 1.3%) implies one month only. The exact months are not stated.
- **Experiment scripts for each table:** the `experiments/` folder exists, but there is no single script or configuration that regenerates Table 3 with its 20 runs.
- **Deployment results:** the Nigerian field test announced for 2020 (p. 9) is not reported in the paper or repository.

7. Reproducibility scorecard

Scoring: Yes = 2, Partial = 1, No = 0 (maximum 18), using the Session 6 checklist.

#	Checklist item	Rating	Justification
1	Seeds reported, many runs?	Partial (1)	20 runs averaged, but no seeds and no SD; unseeded code gives Precision@1% from 15.8% to 34.7%
2	Splits described?	Yes (2)	Temporal split with sizes and illicit ratios per set (p. 6); only the exact validation months are unclear
3	Leakage checked?	Partial (1)	Code is clean (training-only risk features, validation-only thresholds), but illicit rows use post-inspection adjusted values (p. 8)
4	Tuning separated?	Partial (1)	Epoch and threshold chosen on validation; but the hyperparameter curves (pp. 10–11) appear to report test-set metrics, and it is not stated which set guided the defaults
5	Statistical test named?	No (0)	No test for any model comparison
6	Effect size and CI?	No (0)	Point estimates only; no intervals
7	External validation?	Partial (1)	Held-out future year and unseen-importer subgroups (new period); no other country or live deployment results
8	Compute documented?	No (0)	No hardware, run time or exact versions
9	Code and data available?	Partial (1)	Code public with a licence and runs unchanged with 2020-era libraries; but real data under NDA, synthetic data only, dependencies unpinned, and the code fails with current libraries
	Total	**7 / 18 (39%)**	

Overall judgement: partially reproducible (low to moderate). Credit is due for releasing code and a synthetic data set, which is rare in customs research, and for a careful temporal design. Trust in the specific numbers is limited by missing variance, missing statistical tests, an unpinned environment and the post-inspection value issue. The direction of the result (DATE better than tree baselines) is plausible; its exact size is not verifiable.

8. Top 3 recommendations

1. **Control and report randomness.** Fix seeds for XGBoost, PyTorch, NumPy and the data loader; report every metric in Table 3 as mean ± SD (and min–max) over the 20 runs; release the per-run results. This audit verified that a seed option makes each run exactly repeatable (Section 5.2).
2. **Test the comparisons statistically.** Compare DATE with GBDT + LR and TEM over the same runs with a paired test (corrected resampled t-test, or McNemar's test on the fixed test year), and add bootstrap 95% confidence intervals for Precision@1% and Revenue@1%. Report the result computed on the initially declared values as the headline once available, or label Table 3 clearly as using post-inspection values.
3. **Freeze the environment.** Publish a full lock file and Docker image, fix the two deprecated calls (`dt.weekofyear`, `torch.load`), state the hardware and run time, and add one script that regenerates each table from the synthetic data with an expected output, so a stranger can at least verify the pipeline. Section 5.1 shows why: the unpinned environment changed even the deterministic baseline.

Relevance to my thesis

DATE works on the same kind of record as my study: transaction-level import declarations with importer ID, HS code, origin and declared values. Three lessons transfer directly:

- **Split by time and report new importers separately.** I will keep a temporal hold-out (2026) and, inside training, use `StratifiedGroupKFold` with the importer as the group.
- **Never use post-outcome fields.** For my sustainability proxy label, any field filled after inspection or correction would leak the answer.
- **Pin the environment and fix the seeds from day one,** as I did in my `repro-lab` repository: an audit of my own thesis should not find what this audit found.

References

- Kapoor, S., & Narayanan, A. (2023). Leakage and the reproducibility crisis in machine-learning-based science. *Patterns, 4*(9), 100804. https://doi.org/10.1016/j.patter.2023.100804
- Kim, S., Tsai, Y.-C., Singh, K., Choi, Y., Ibok, E., Li, C.-T., & Cha, M. (2020). DATE: Dual attentive tree-aware embedding for customs fraud detection. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining* (pp. 2880–2890). ACM. https://doi.org/10.1145/3394486.3403339
- Nadeau, C., & Bengio, Y. (2003). Inference for the generalization error. *Machine Learning, 52*, 239–281.
- Repository audited: Roytsai27/Dual-Attentive-Tree-aware-Embedding, commit `5c540d6`.
- Kapoor, S., Cantrell, E. M., Peng, K., Pham, T. H., Bail, C. A., Gundersen, O. E., Hofman, J. M., Hullman, J., Lones, M. A., Malik, M. M., Nanayakkara, P., Poldrack, R. A., Raji, I. D., Roberts, M., Salganik, M. J., Serra-Garcia, M., Stewart, B. M., Vandewiele, G., & Narayanan, A. (2024). REFORMS: Consensus-based recommendations for machine-learning-based science. *Science Advances, 10*(18), eadk3452. https://doi.org/10.1126/sciadv.adk3452
- Audit materials (patches, scripts, logs): github.com/angelaacosta-byte/date-repro-audit